

PROJET DE GROUPE



Weather App — consommer une API REST avec formulaire & routing

Durée 1 journée et demie	Groupe 4 personnes	Soutenance 15 minutes / groupe
------------------------------------	------------------------------	--

SOMMAIRE

- | | |
|--------------------------------------|--|
| 1. Contexte et objectifs | 10. Gestion du chargement et des erreurs |
| 2. Page d'accueil | 11. Carte blanche |
| 3. Formulaire de recherche | 12. Fonctionnalité libre |
| 4. Routing | 13. Postman |
| 5. Affichage de la météo | 14. README attendu |
| 6. API OpenWeather | 15. Livrables |
| 7. Service Angular et gestion d'état | 16. Travail en groupe |
| 8. Composants Angular | 17. Présentation finale |
| 9. Communication entre composants | 18. Évaluation |

1. CONTEXTE ET OBJECTIFS

Vous allez réaliser en groupe une application web de météo avec Angular et l'API OpenWeather. L'objectif est de mettre en pratique les notions fondamentales du cours :

- composants Angular, formulaires, routing, services
- communication entre composants, gestion d'état
- requêtes HTTP, API REST, données JSON
- gestion des erreurs et des états de chargement

Fonctionnement général attendu

```
Utilisateur
| recherche une ville
v
Formulaire Angular (Reactive Forms)
v
Routing
v
Composant Angular
v
Service Angular
| HTTP GET
v
OpenWeather API
| JSON
v
Service Angular -> Composant -> Interface utilisateur
```

2. PAGE D'ACCUEIL

Créer une page d'accueil accessible avec une route dédiée : **/home**. Cette page doit :

- présenter votre application
- permettre à l'utilisateur de rechercher une ville

- contenir le formulaire de recherche

Le contenu et le design sont libres.

Maquette indicative — page d'accueil

■ Indication — couleurs, typographie et disposition exacte restent entièrement libres (carte blanche, voir section 11). Seuls les éléments structurels sont attendus : une barre de navigation, un titre d'accroche, un formulaire de recherche avec champ + bouton, et une zone affichant soit un état vide, soit le résultat de la recherche une fois la ville validée.

3. FORMULAIRE DE RECHERCHE

La recherche doit obligatoirement être réalisée avec un formulaire Angular, comportant au minimum :

- un champ de saisie pour la ville
- un bouton de recherche

```
+ ----- +
| Rechercher une ville |
| |
| [ Paris ] |
| |
| [ Rechercher ] |
+ ----- +
```

■ Indication — le champ doit être obligatoire — si l'utilisateur valide un formulaire vide, un message doit être affiché : « Veuillez saisir une ville. » Vous devez utiliser les mécanismes de formulaires Angular vus en cours (Template-driven ou Reactive Forms).

4. ROUTING

Votre application doit utiliser le routing Angular. Au minimum, les routes suivantes doivent être présentes :

```
/home
/weather/:city
/about
```

- **/home** — page d'accueil et formulaire de recherche
- **/weather/:city** — page affichant la météo de la ville recherchée (ex. /weather/Paris, /weather/Lille, /weather/Tokyo). La ville doit être récupérée depuis le paramètre de route
- **/about** — page présentant votre application, les membres du groupe, les technologies utilisées

Cette structure sépare volontairement trois responsabilités distinctes : la saisie (/home), l'affichage d'une ressource identifiée par un paramètre dynamique (/weather/:city), et une page statique d'information (/about). C'est l'occasion de manipuler le paramétrage de route (ActivatedRoute) et de comprendre pourquoi la ville transite par l'URL plutôt que par une variable de composant.

5. AFFICHAGE DE LA MÉTÉO

Pour chaque ville recherchée, afficher au minimum :

- nom de la ville
- pays
- température actuelle
- température ressentie
- description météo
- humidité
- vitesse du vent
- icône météo

```
+ ----- +
| Paris (FR) |
| |
| 24 C |
| Ciel degage |
| |
| Ressenti : 25 C |
| Humidite : 58 % |
| Vent : 12 km/h |
+ ----- +
```

■ **Important — les données doivent provenir de l'API. Aucune donnée météo ne doit être codée en dur.**

■ *Indication — la température doit être affichée en degrés Celsius. La réponse brute de l'API OpenWeather renvoie les températures en Kelvin — à vous de trouver et justifier la manière dont vous gérez cette conversion dans votre application.*

6. API OPENWEATHER

Vous devez utiliser OpenWeather. Vous devrez :

- créer un compte
- récupérer votre clé API
- consulter la documentation
- comprendre l'endpoint utilisé
- tester vos requêtes avec Postman
- analyser la réponse JSON
- identifier les données nécessaires
- intégrer la requête dans Angular

Vous devrez être capables d'expliquer : URL + Méthode HTTP + Paramètres + API Key → Réponse JSON

Email de confirmation et exemple de requête

Une fois votre compte créé, OpenWeather vous envoie un email de confirmation contenant votre clé API ainsi qu'un exemple de requête à utiliser. L'endpoint de base est **api.openweathermap.org**. Exemple de requête pour la météo actuelle d'une ville :

```
GET https://api.openweathermap.org/data/2.5/weather?q=Paris&APPID={votre_cle_api}
```

Pour la météo actuelle, cette requête vous suffit. Pour toute fonctionnalité supplémentaire nécessitant un autre type de données (prévisions, géolocalisation, historique...), c'est à vous de consulter la documentation OpenWeather et d'identifier l'endpoint, les paramètres et le format de réponse correspondants — cela fait partie du travail attendu et sera à expliquer en soutenance.

Compte, clé API et limites d'utilisation

- rendez-vous sur openweathermap.org et créez un compte gratuit (aucune carte bancaire requise)
- récupérez votre clé depuis l'onglet « API keys » de votre compte
- un seul compte et une seule clé API par groupe suffisent — ne créez pas une clé par membre

■ *Indication — une clé API OpenWeather n'est pas active immédiatement après sa création — l'activation peut prendre jusqu'à 2 heures, pendant lesquelles vos requêtes échoueront avec une erreur 401 Unauthorized même si la clé copiée est correcte. Créez votre compte et générez votre clé la veille du projet, ou au minimum 2 heures avant le début de la séance. En attendant, vous pouvez avancer sur la structure Angular (composants, routing, formulaire) avec des données factices.*

Le plan gratuit OpenWeather permet 60 appels par minute et jusqu'à 1 000 000 d'appels par mois. Au-delà du seuil autorisé, OpenWeather renvoie une erreur 429 Too Many Requests.

■ *Indication — votre application ne doit pas générer d'appels API inutiles ou excessifs, y compris en cas de frappe rapide dans le champ de recherche ou de recherches répétées sur la même ville. Vous devrez présenter et justifier votre approche technique en soutenance, ainsi que les alternatives que vous auriez pu envisager.*

Sécurité de la clé API

- ne jamais pousser votre clé API en clair sur Git (dépôt public ou privé)
- stockez-la dans un fichier de configuration (ex. environment.ts) et ajoutez-le à votre .gitignore
- dans votre README, expliquez comment configurer sa propre clé sans jamais y faire apparaître la clé réelle du groupe

7. SERVICE ANGULAR ET GESTION D'ÉTAT

Les appels à l'API doivent être réalisés dans un service Angular dédié (ex. weather.service.ts). Les composants ne doivent pas effectuer directement les appels HTTP.

```
WeatherComponent -> WeatherService -> OpenWeather API
OpenWeather API -> WeatherService -> WeatherComponent
```

Vous devez utiliser HttpClient.

Gestion de l'état

L'état de l'application (ville courante, données météo, chargement, erreur) doit être géré avec les mécanismes vus en cours : signals et/ou observables RxJS. Vous êtes libres d'organiser cet état comme vous le souhaitez, mais vous devrez justifier votre choix en soutenance, notamment sur la manière dont vous évitez les problèmes de rafraîchissement de template après un appel asynchrone.

8. COMPOSANTS ANGULAR

Votre application doit être composée de plusieurs composants. Vous devez notamment séparer les responsabilités liées :

- à la navigation
- à la recherche
- à l'affichage météo
- aux différentes pages

```
app/
components/
navbar/
search/
weather/
pages/
home/
weather/
about/
services/
weather.service.ts
```

Cette architecture est un exemple et non une obligation.

9. COMMUNICATION ENTRE COMPOSANTS

Vous devez utiliser les mécanismes Angular appropriés lorsque plusieurs composants doivent communiquer : @Input, @Output, événements, services. L'objectif est de comprendre comment organiser les échanges de données entre les différentes parties de votre application.

10. GESTION DU CHARGEMENT ET DES ERREURS

Pendant une requête vers l'API, l'utilisateur doit être informé qu'une opération est en cours (ex. « Chargement de la météo... »), avant, pendant et après la requête.

Votre application doit gérer au minimum :

- Formulaire invalide → « Veuillez saisir une ville. »
- Ville inexistante → « Ville introuvable. »
- Erreur API → « Impossible de récupérer les données météo. »
- Trop de requêtes (429) → « Trop de requêtes, veuillez réessayer dans quelques instants. »

■ **Important** — l'application ne doit pas se retrouver dans un état incohérent lorsqu'une erreur survient.

11. CARTE BLANCHE

Vous avez carte blanche sur le style et l'architecture. Le cahier des charges définit ce qui doit fonctionner, pas comment votre application doit être construite visuellement ou techniquement.

- **Design** — couleurs, typographie, mise en page, animations, icônes, thème clair/sombre, illustrations
- **Architecture** — organisation des dossiers, découpage des composants, organisation des services, gestion des données
- **Technologies complémentaires** — bibliothèques supplémentaires autorisées, à condition de justifier vos choix

12. FONCTIONNALITÉ LIBRE

Chaque groupe doit ajouter au moins une fonctionnalité non demandée explicitement dans le cahier des charges.

Exemples :

- favoris
- géolocalisation
- prévisions
- historique
- localStorage
- dark mode
- graphique
- actualisation automatique
- comparaison de villes
- recommandations vestimentaires
- interface dynamique selon la météo

Vous pouvez proposer votre propre idée. La fonctionnalité doit s'appuyer sur un **second endpoint** de l'API OpenWeather (par exemple les prévisions à 5 jours) plutôt que de se limiter à un ajout purement visuel ou local (CSS, localStorage seul). Elle doit être pertinente et présentée lors de la soutenance.

Exemple indicatif — prévisions à 5 jours

```
+ ----- +
| Previsions - Paris (FR) |
| |
| Jeu 21 Ven 22 Sam 23 Dim 24 Lun 25 |
| 19C 21C 18C 17C 20C |
| Nuageux Soleil Pluie Pluie Soleil |
+ ----- +
```

■ *Indication* — maquette purement indicative — la mise en forme reste libre (carte blanche, voir section 11). Elle illustre uniquement le niveau d'information attendu si vous choisissez cette fonctionnalité.

13. POSTMAN

Avant d'intégrer l'API dans Angular, vous devez tester et comprendre vos requêtes avec Postman. Vous devez fournir une collection Postman dédiée à votre projet :

```
Weather App
Current Weather
Paris
Lille
Tokyo
Forecast (si fonctionnalité libre associée)
Tests
```

Variables possibles : `{{base_url}}`, `{{api_key}}`, `{{city}}`. Chaque requête importante doit être documentée : objectif, méthode HTTP, URL, paramètres, headers éventuels, réponse attendue, données importantes récupérées. Votre collection doit permettre à une autre personne de comprendre vos requêtes sans avoir besoin de vous demander d'explication.

14. README — DOCUMENTATION ATTENDUE

Votre projet doit obligatoirement contenir un fichier README.md, complet, clair et professionnel, avec les sections suivantes :

- Présentation — nom du projet, objectif, présentation rapide de l'application
- Membres — nom des membres du groupe
- Technologies — Angular, TypeScript, HTML/CSS, OpenWeather API, Postman
- Installation — comment installer et lancer le projet (npm install, ng serve)
- Configuration — comment configurer la clé API — ne jamais publier la vraie clé
- Fonctionnalités obligatoires — listez les fonctionnalités réalisées
- Fonctionnalités supplémentaires — présentez votre fonctionnalité libre
- Architecture — expliquez votre organisation Angular, éventuellement avec un schéma
- API — l'API utilisée, les endpoints, les paramètres, les données récupérées
- Postman — comment utiliser votre collection Postman
- Difficultés rencontrées — au moins deux difficultés rencontrées et leur résolution
- Améliorations possibles — ce que vous auriez voulu améliorer avec davantage de temps

15. LIVRABLES

- Projet Angular — projet fonctionnel respectant le cahier des charges
- README.md — documentation complète et claire
- Collection Postman — collection fonctionnelle et documentée
- Configuration nécessaire — fichiers nécessaires au fonctionnement, indiqués dans le README

■ **Important — aucune clé API ou donnée sensible ne doit être déposée dans Git.**

16. TRAVAIL EN GROUPE

Vous êtes libres de vous organiser. Cependant :

- chaque membre doit participer au projet et contribuer au code
- chaque membre doit comprendre une partie du projet
- chaque membre doit pouvoir expliquer les choix réalisés

La répartition des tâches est libre. L'historique de commits du dépôt Git doit permettre d'identifier la contribution de chaque membre ; il pourra être consulté lors de la soutenance.

17. PRÉSENTATION FINALE — 15 MINUTES PAR GROUPE

- **1 — Démonstration (≈ 7 min)** — navigation, formulaire, recherche d'une ville, affichage météo, gestion d'une erreur, fonctionnalité supplémentaire
- **2 — Apprentissages (≈ 4 min)** — composants, formulaires, routing, services, HttpClient, API REST, JSON, communication entre composants, TypeScript, Postman
- **3 — Ce qui pourrait être amélioré (≈ 4 min)** — difficultés rencontrées, choix réalisés, points faibles actuels, fonctionnalités à ajouter, ce que vous feriez différemment avec plus de temps

18. ÉVALUATION

Critère	Points
Respect du cahier des charges	5 pts
Angular : composants, services, formulaires, routing	4 pts
Gestion de l'état (signals/observables) + justification des choix	2 pts
Communication avec l'API	2 pts
Gestion erreurs / chargement	1 pt
Postman + documentation API	2 pts
README	1 pt
Présentation orale	3 pts
TOTAL	20 pts

Bonus / appréciation — la fonctionnalité libre et les initiatives particulières pourront être valorisées dans l'évaluation de la qualité du projet.

CONSIGNE FINALE

Le cahier des charges est obligatoire. Le style et l'architecture sont libres. Vous devez faire des choix, les justifier et être capables d'expliquer votre code. Une application simple, propre, fonctionnelle et bien comprise est préférable à une application très ambitieuse que vous ne maîtrisez pas. Votre objectif n'est pas seulement de faire fonctionner l'application : vous devez comprendre comment Angular communique avec une API REST, et être capables de justifier chacun de vos choix techniques.

RESSOURCES

- OpenWeather — documentation API
- OpenWeather — FAQ (activation de clé, limites)
- Angular — documentation Signals
- Postman — documentation officielle et guide des collections